Djedefre

ljm

1. Intro

Djedefre is a tool for documenting your network.

Djedefre consists of 3 parts:

- the database
- scan scripts
- the GUI/webserver

The scan scripts fill the database. The GUI displays the network in the database and allows the user to change certain values. The database is created when the GUI is first started.

Some time ago, there was a tool called Cheops, that automated network discovery and gave a nice drawing of the network. Cheops is now abandonware. Djedefre was pharaoh after Cheops. The main difference between Cheops and Djedefre is, that Djedefre assumes that the network is managed. This means that there is access to at least some servers, routers and switches.

2. Using Djedefre

2.1 Installing

From "https://github.com/ljmdullaart/djedefre" download at least

- dancr.pl
- api_routes.pm
- common_functions.pm
- dje_db.pm
- djedefre_user.pm
- drawing.pm
- djedefre_create_db.pl
- the directory public
- the directory views
- the directory scan_scripts

Make sure that all the prerequisites in appendix A are met.

2.2 First start

After installing or unpacking,

- Move to the directory where Djedefre is installed.
- Make sure that there is a directory database and create it if it is not there
- Create the database with perl djedefre_create_db.pl
- start the webserver with nohup ./dancr.pl &

With a web browswer, navigate to "http://127.0.0.1:3000"

You will be greeted with an empty page, with in the top-right a log in button

Click the login, and a login screen will appear.

The cartouche depicted is Djedefre. You can now login. Any user that is defined on the system will be granted access.

At this moment, Djedefre does not contain any data. Exploring further is therefore useless.

2.3 Concepts

The heart of Djedefre is the database. Most information that is displayed comes from the database. Also, many scripts rely on information from the database.

The user interface is handled by the dancer2 webserver that was started with `nohup ./dancr.pl &` The dancer2 application uses the information from the database to display in various forms, from lists to drawings. The drawings are also automatically saved.

The database is filled with scan scripts. Scan scripts are stored in the directory `scan_scripts`. There are different type of scan scripts: for filling the database and scripts that produce output on stdout. The scripts that may update the database all start with `scan_` for scripts that scan the network or with `page_` for scripts that generate standard pages, or network drawings.

There is an optimal sequence for running the `scan_` and `page_` scripts. The script `right_sequence.sh` runs that sequence.

The other scan scripts that start with `dashboard_` or `listing_` or `status_` are used to create an operational dashboard.

2.4 Scan the network

Scan the network by running

```
bash right_sequence.sh
```

in the scan script directory. It helps if you have password-less `sudo` rights. When the scan finishes, reload the web page in the browser. Under the menu

--- Select Drawing ---

there ar now a number of drawings that correspond to the `pages_` scripts. Select the option

all_subnets

and a drawing with all induvidual subnets appear. The subnets that you see are the subnets that are connected to the local system. If you click on a subnet, the pane on the right side will display some additional information. For the scanning, the Scope is important. This is a drop-down list with the following options:

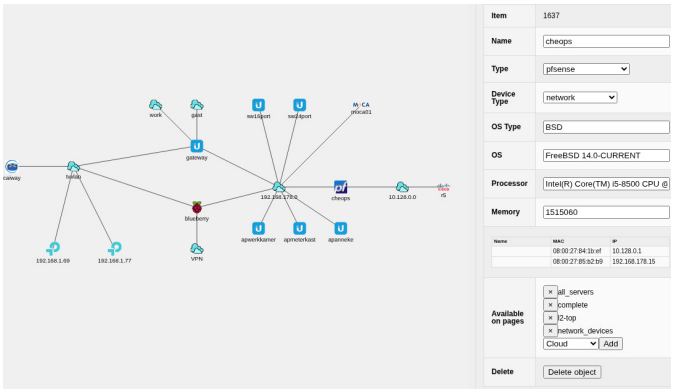
(empty)	The subnet is not under control and will not be scanned
global	The subnet is a global network, but under our control. It will be scanned
local	The subnet is a local subnet that must be scanned.
server	A subnet that is specific for a server, for example 127.0.0.0/8. It will not be scanned.
shared	A shared network that must not be scanned

For your local network, select "local" and run the scan `right_sequence.sh` again. You should now see other hosts as well in the "all_server" drawing. In the "complete" you will see a start of a drawing off your network.

Running `right_sequence.sh` will make the drawing more complete. If you have set-up passwordless ssh login to other servers on your network, Djedefre will try to obtain information from those servers or devices. That may lead to the discovery of other subnets.

2.5 Documenting the network

If there is access to the different devices the scanning scripts will collect information from those devices. After a few iterations of scanning and classifying networks, the network may look something like below.



When a device or subnet is clicked, the pane on the right will show the details of that device.

Many fields can be edited. For the text fields, enter commits the changes. For the selection fields, a new seletion is automaticaly committed.

Below there is a selection table for the pages on which the object should be available. And below that, there is a button that can delete the object from the database.

Objects can be relocated in the drawing with drag and drop.

2.6 Types of objects

2.6.1 Servers

Servers are the main objects in Djederfre. A server is a device that is connected to the network. When scanning, the scan scripts try to find out as much as possible about a server. That includes DNS name and, if a login is possible, CPU type, OS and so on. If the sanning process has ssh access to the server, it will have more information.

Servers are connected to the network with interfaces. Interfaces are not separate objects.

Item	Item number in the database
Name	Name of the device; can be edited
Type	Type. Select from drop-down list. Determines the icon in the drawing.
Device Type	Device type is a general classification of the device
OS Type	Type of OS
OS	Specific OS; version and release
Processor	The processor in the device
Memmory	The quantity of memmory in kB
Interfaces	A table of detected interfaces

2.6.2 Subnets

Subnets are IPv4 subnets. They are defined by the base address and the number of bits in the network mask, called cidr. Networks have a scope, that determines whether they should be scanned. Networks without scope will not be scanned. A desctiption of the scope has been given above.

Item	Item number in the database
Name	Name of the subnet; can be edited
Type	Type. Select from drop-down list. Determines the icon in the drawing.
Network address	Base address of the network
CIDR	number of network bits
Scope	Scope can be empty, global, local, server or shared

2.6.3 Cloud

Clouds are services on the Internet. Djedefre does not use private clouds: private clouds are just local servers. Like servers, clouds can have multiple types.

Item	Item number in the database
Name	Name of the subnet; can be edited
Type	Type. Select from drop-down list. Determines the icon in the drawing.
Vendor	The vendor of the cloud service
Service	description of the cloud service

2.7 Lists

In the menu, there is an option to select a list. The lists are provided for the documentation of the network. They are basically dumps of the database, though some combinations are made to make the content more usefull.

2.8 Status

The statusses accumulate the results of various custom scripts. A separate chapter is dedicated to the statusses.

3. Scan scripts

3.1 Intro

Scan scripts are under the directory `scan_scripts.` The scripts try to map the network and devices connected to the network. They may be started in a random order, but the script `right_sequence.sh` starts them in an order that is most efficient on my personal network.

The scripts update the database with their findings. You may need to reload the web page in the GUI to have the results displayed.

3.2 Scan_ srctipts

3.2.1 local_system

Scans the local system. The local system is added to the database if it is not there yet and all the interfaces and the subnets to which they connect are added.

3.2.2 subnet

All subnets for which the scope is global or local are scanned with `fping` and all interfaces are added to the database if they are not there yet. For interfaces that do not yet have a server attached, a new server is created.

3.2.3 access

Scans the known servers for access. It tries to access the server with ssh, first without specifying a user, then as root and finally as admin. If you have the program `dotelnet` installed, then `dotelnet` is also used to try to gain access to the server. Access is verified by trying an echo or trying `sh ip int br`

3.2.4 arp

Retrieves the ARP tables from all servers that it has access to. If the IP address in the ARP table is not yet known, an interface, and possibly a server, is created. For all interfaces that are found, the MAC-id is added.

3.2.5 type

Tries to determine the type of the server. For servers with access, the existence and contents of files and/or the output of simple commands is used. For other servers, AVAHI is used.

3.2.6 mergeif

Merges interfaces to servers. Some servers have different network interfaces, and the scanning process may create a different server for each interface. If the scan script can determine that an interface belongs to another server, the servers are merged.

3.2.7 server

This works like `scan_local_system.sh` but for remote servers where access is possible.

3.2.8 serverinfo

For servers with access, tries to determine as much information about the server as possible.

3.2.9 vbox

3.2.10 internet

3.2.11 arp

3.2.12 l2scans

3.2.13 data

3.2.14 clean_srv

4. Status

4.1 Intro

The statusses display the result of various scripts. That may be a a listing, a status (up/down) or a value or percentage. The default scanning for up/down is a ping. However, if a script `status_<servername>.sh` is present, that script will be used to detemine the status.

The dashboard displays 4 columns:

— The status of servers, which may be up, down or exluded. The list is sorted; all down servers are at the top and all excluded are at the bottom

— The status of the network. At the top is the availability of the Internet and the route from the scanning device towards the Internet, and below the status of all network devices sorted in the same maner as the servers.

— Statusses of specific items. These are generated by scripts under `scan_scripts` that start with `dashboard_`

— A set of textual listings. These are generated by scripts under `scan_scripts` that start with `listing_` when they are called with `-s` as argument.

Listings provide a tile per listing script. For the tiles, the listing script is called with `-htm` as argument.

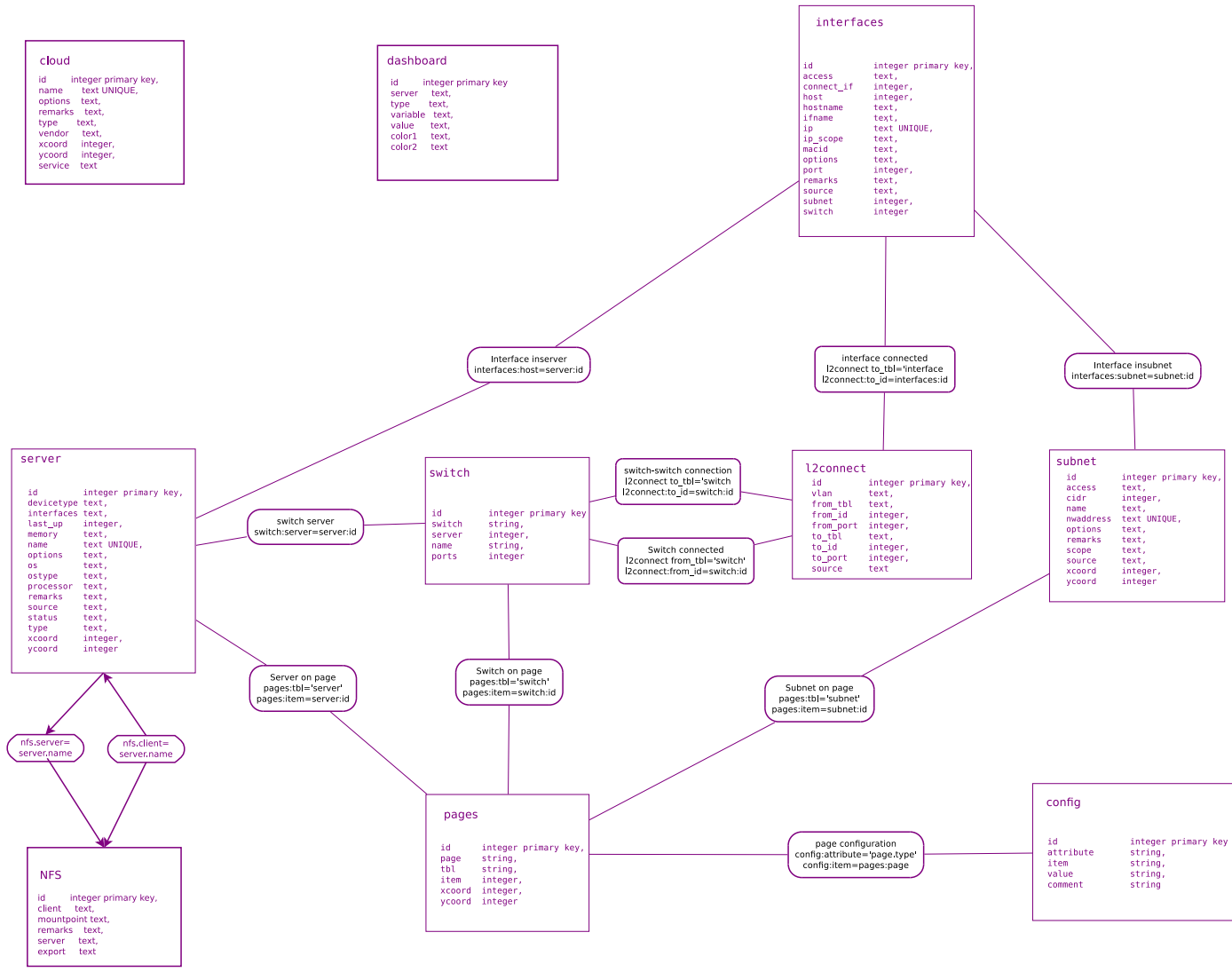
4.2 Status scripts

A status script detemines the status of a server. If the status is up, the script must end with an exit status 0. If the status is down, an exit status of 1 is expected. The scripts should be kept simple and should match your environment. As an example, the following script tests is the Citadel server is running on a host:

```
#!/bin/bash

if ssh aesopos ps -ef | grep -q 'citserver' ; then
          exit 0
else
          exit 1
fi
```

5. Database



Most fields are only filled-in if they are known

5.1 interfaces

id	integer primary key autoincrement	ID of the interface
macid	string	MAC-ID
ip	string	IP address
hostname	string	
host	integer	
subnet	integer	
access	string	
connect_if	integer	
port	integer	

5.2 subnet

id	integer primary key autoincrement
nwaddress	string
cidr	integer
xcoord	integer
ycoord	integer
name	string
options	string
access	string

5.3 server

id	integer primary key autoincrement
name	string
xcoord	integer
ycoord	integer
type	string
status	string
last_up	integer
options	string
ostype	string
os	string
processor	string
devicetype	string
memory	string

5.4 command

id	integer primary key autoincrement
host	string
button	string
command	string

5.5 details

id	integer
type	string
os	string

5.6 pages

id	integer primary key autoincrement
page	string
tbl	string
item	integer
xcoord	integer
ycoord	integer

5.7 switch

id	integer primary key autoincrement
switch	string
server	integer
name	string
ports	integer

5.8 l2connect

id	integer primary key autoincrement
vlan	string
from_tbl	string
from_id	integer
from_port	integer
to_tbl	string
to_id	integer
to_port	integer

5.9 config

id	integer primary key autoincrement
attribute	string
item	string
value	string

5.10 cloud

id	integer primary key autoincrement
name	string
vendor	string
type	string
xcoord	integer
ycoord	integer
service	string

5.11 dashboard

id	integer primary key autoincrement
server	string
type	string
variable	string
value	string
color1	string
color2	string

5.12 nfs

id	integer primary key autoincrement
server	string
export	string
client	string
mountpoint	string

6. The source code

If I want to change anything, I find myself first digging through the source code before I can even begin to add new functionality. This chapter should help finding my way through the code easier.

Djedefre is a Dancer2 fat app. The app is divided in two parts:

— the Dancer2 erl application

— the javascripts

and the views that glue them together.

6.1 Dancer2 app

Everything is serverd by the Dancer2 app `dancr.pl`. The app is somewhat modularized. The main app file contains the routes that server the HTML pages using the template_toolkit. API calls are served by the routes in `api_routes.pm`. All user management is done by `djedefre_user.pm` including the routes needed for that. All database access is centralized in `dje_db.pm`. If someone wants to change the underlying database, only this file (and its equivalent for the scan scripts) need to be changed. The drawing of a network plot is a somewhat complex task. This is separated in the file `drawing.pm`. Finally, some common functions in `common_functions.pm` are used in different parts of the app.

APPENDIX A

Prerequisites

A.1 Packages

— perl
— cpanminus
— build-essential
— libssl-dev
— curl
— libexpat1-dev
— libpcap-dev
— libmagickcore-dev
— imagemagick
— libimage-magick-perl
— sqlite3
— cron
— openssh-server
— telnetd
— inetutils-inetd
— nmap
— iproute2
— ipcalc
— shared-mime-info
— iputils-ping
— arping
— avahi-utils
— bsdextrautils
— curl
— dos2unix
— fping
— grepcidr
— bind9-host
— jq
— sqlite3

- openssh-client
- sudo
- groff-base
- wget
- libauthen-simple-pam-perl

A.2 Perl CPAN

- DBI
- DBD::SQLite
- Dancer2
- Dancer2::Plugin::Database
- File::HomeDir
- File::Slurp
- File::Slurper
- Image::Magick
- List::MoreUtils
- Net::DHCP::Constants
- Net::DHCP::Packet
- Net::Pcap
- Net::RawIP
- Sort::Naturally
- Template
- XML::Parser
- Dancer2::Plugin::Auth::Tiny
- Crypt::Argon2

CONTENTS